

Description:

Flowdometer is a free, open-source SLA engine that bolts onto any standard or custom object in Salesforce. Instead of relying on the Case-only Entitlements/Milestones framework, it watches whichever fields you already have history-tracking enabled for—stage, status picklists, owner look-ups, even custom references—and records each transition as a "Step" tied to a parent "Flow."

An Apex service layer crunches the heavy lifting (timestamps, business-hour math, goal variance), while a bundle of pre-built dashboards surfaces cycle-time, first-response, idle-time and breach analytics out-of-the-box.

Implementation is intentionally lightweight: install the managed package, pick the object/field pair you care about, and Flowdometer starts stamping elapsed-time metrics—no Entitlement model, no support process, no extra configuration on Cases.

Because everything is stored in native custom objects, admins can extend the schema or reference the data in Reports, Flows, or SOQL just like any other record. Typical adopters wire it to post-sale operations objects (onboarding projects, order fulfilment records, ticket escalations) so they can quantify bottlenecks, benchmark teams, and prove SLA compliance alongside core CRM data.

In short, Flowdometer gives you Milestones-style tracking for every object, minus the setup drag and object restrictions.

Marketing Site:

<https://www.flowdometer.com/>

End User Setup Documentation:

<https://www.flowdometer.com/documentation>

Security Measures:

<https://raw.githubusercontent.com/michaelmuse/Flowdometer/refs/heads/master/SecurityMeasures.txt>

Data Model

https://gist.githubusercontent.com/michaelmuse/ef3ccbd0692b0c633f5428168442dd9f/raw/6307d878ee2af66961616bc658f25d1fbce03520/gist_content.json

Apex

All Method signatures in my apex files:

<https://gist.githubusercontent.com/michaelmuse/c29b1571124939fa60fc0b26df>

14bba2/raw/acfd932c08c7eaa82cad03f44088426977e1649b/FlowdometerApexMethodSignatures.json

ListenerFlowController.cls

Latest code:

<https://gist.githubusercontent.com/michaelmuse/cb4df0a6a0c1cbf630e81cf28c70000f/raw/fc2f1c06093cab32555b00b60ea2e818d2df56b3/ListenerFlowController.cls>

Main Invocable Action: Query & Parse History Records

Used by Flow 2

ListenerFlowControllerTest.cls

Latest code:

<https://gist.githubusercontent.com/michaelmuse/6908a3176b423bee76b216ab33948492/raw/c2elbbfeaccaaccfaac00583a5f97d78158f9877/ListenerFlowControllerTest.cls>

Tests our controller, ListenerFlowController.cls

TestDataFactory.cls

Latest code:

<https://gist.githubusercontent.com/michaelmuse/6ff9d325ef68cf17d3c0688fc4461ala/raw/92eb92bc3342a28a3bf401372075551b85a4e9d6/TestDataFactory.cls>

Builds data for ListenerFlowControllerTest.cls

ListenerUpdateFlowController.cls

Main Invocable Action: Update Flow Records

Used by Flow 3

Defined in ListenerUpdateFlowController.cls

Flows

Flow 1 (Flowdometer - Listener Tracking Trigger):

Latest code:

https://gist.githubusercontent.com/michaelmuse/0bb5a6600c34799382b42c1998e9fbf9/raw/48ec3cb2a762beabc1470e6787ed06ca83576fc9/Listener_Batch_Flow.flow-meta.xml

Name: Flowdometer - (1) Listener Tracking Trigger

Description: Flowdometer handler for setting up listeners for your object and field.

Trigger Conditions

Object: Listener__c

Trigger Type: Record After Save

Record Trigger Type: Create and Update

Additional Filters: isActive__c should be equal to true

Elements

Decision Element: "Check If the Flow is Active"

Checks if Flowdometer__isActive__c field on the record is true.

If true, moves to assignment element.

Assignment Element: "FlowdoConfig Value Assignment"

Assigns the current record to a variable varFlowdoConfig.

Sub-Flow: "Listener Config Main Flow"

Takes the varFlowdoConfig as an input argument.

Scheduled Paths

Run Every 3 Minutes: The flow runs every 3 minutes, depending on the Last_Check__c field of the record.

Context of Loops and DMLs

Loops: No loops detected.

DML Operations: No explicit DML operations detected, but the sub-flow "Listener Config Main Flow" may have DML operations (we can't tell from this XML alone).

Efficiency and Bulkification

Efficiency: As far as we can tell from this XML, the flow seems relatively straightforward. It doesn't contain any loops or DML operations, which are usually the key factors affecting bulkification.

Bulkification: Since the flow triggers after record creation and update, and it doesn't show any DML or loop operations, it should generally be bulk-safe. However, the sub-flow might be a point of concern for bulkification.

Flow 2 (Flowdometer - Listener Finds Changed Records):

Latest code:

https://gist.githubusercontent.com/michaelmuse/8bd43794f17e72112704184ee04ddd7b/raw/b7b003785033b8a581d60acaafe51e460f9c0635/Listener_Configuration_Main_Flow.flow-meta.xml

Name: Flowdometer - (2) Listener Finds Changed Records

Description:

Triggered by: Flowdometer - Listener Tracking Trigger.
Flowdometer handler to query for untracked changes.

Trigger Conditions: This appears to be an AutoLaunched Flow, likely called from another flow or process.

Elements

Decision Element: "Check FlowdoConfig"

Checks if varFlowDoConfig.isActive__c is true.

If true, moves to record lookup.

Record Lookup: "Get ALL Record Types"

Queries for specific RecordType objects.

Decision Element: "Found Record Types?"

Checks if any records were found in the previous step.

New: Apex Action: "Sort by Record Type"

Calls an Apex action to sort records by their Record Type.

Apex Action: "Query and Parse History Records Latest"

Calls an Apex action ListenerFlowController and passes varFlowDoConfig as an input parameter.

Decision Element: "Check Records"

Checks the output from the Apex action to see if there are records (hasRecords) and if the check was successful (checkSuccess).

Loop: "Loop Through Parsed Records"

Iterates through lstListenerFlow.

Sub-Flow: "Create Tracker Records Using Parsed History Records"

Called within the loop to process each record.

Assignment Elements:

Multiple assignment elements to handle error messages and update Last_Check__c and Last_Execution_On__c.

Record Update: "Update FlowdoConfig"

Updates the Listener__c record.

Context of Loops and DMLs

Loops: One loop, "Loop Through Parsed Records," iterating through lstListenerFlow.

DML Operations: Two explicit record update operations, plus the DMLs that might be in the sub-flow and Apex action.

Efficiency and Bulkification

Efficiency: This flow is more complex than the first one and includes Apex actions for sorting, loops, and multiple decision and assignment elements.

Bulkification: The flow contains a loop and multiple DML operations, which could be points of concern for bulkification.

Flow 3 (Flowdometer - Listener Creates Trackers):

Latest code:

https://gist.githubusercontent.com/michaelmuse/0cb1f81bb578cc10cd4f9c0189cc1793/raw/9617ea11496613561986f413c8f4d0771808adce/Listener_Flow_Sub_Flow.flow-meta.xml

Name: Flowdometer - (3) Listener Creates Trackers

Description: This flow is used to create Flow and Step records for the record being tracked, using history records collected in the Flowdometer - (2) Listener Finds Changed Records flow.

Triggered By: This is an AutoLaunched Flow, likely called from another flow or process.

Elements:

Record Lookups: There are three record lookup elements that query for specific Flow__c and Step__c objects based on certain conditions.

Decisions: There are multiple decision elements that check for the existence of Flows, Steps, and specific record types. They also check if certain records were found and decide the next steps based on these conditions.

Apex Actions: There is one Apex action that calls the ListenerUpdateFlowController and passes various parameters as input.

Assignments: Multiple assignment elements are used to set error messages, set up Flow Tracker, set Goal Flow, set Tracker Flow, set Goal ID, set Latest Tracker Step, and more.

Sub-Flows: There are no sub-flows in this flow.

Loops: There are three loops that iterate through the Get_ALL_Steps and varALLFlows collections and sort and assign flows.

Record Creates: Two record creates elements are used to create Flow Tracker and Step.

Record Updates: Multiple record updates are used to add error messages to the Listener, update Flow, and update Last Step.

Context of Loops and DMLs: Loops: Three loops, iterating through Get_ALL_Steps, varALLFlows and Sort_and_Assign_Flows. DML Operations: Two explicit record create operations and multiple record update operations.

Efficiency and Bulkification: Efficiency: This flow is complex with multiple decision, assignment, record lookup, record create, record update elements and loops. Bulkification: The flow contains multiple loops and DML operations which could be points of concern for bulkification.